Flutter Fundamentals –
User interface – Navigation and Routing

Contents

1. User interface 2
   1.1. Navigation and Routing 2
      1.1.1. Add tabs to your app 2
      1.1.2. Navigate to a new screen and back 4
      1.1.3. Send data to a new screen 6
      1.1.4. Return data from a screen 10
      1.1.5. Add a drawer to a screen 13
      1.1.6. Animate a widget across screens 16
2. Thực hành 18
   Bài tập 1. Login and Registration 18
   Bài tập 2. Login and Registration với GoRouter package (đọc thêm) 32
   Bài tập 3. How to Get Started With GoRouter in Flutter (đọc thêm) 40

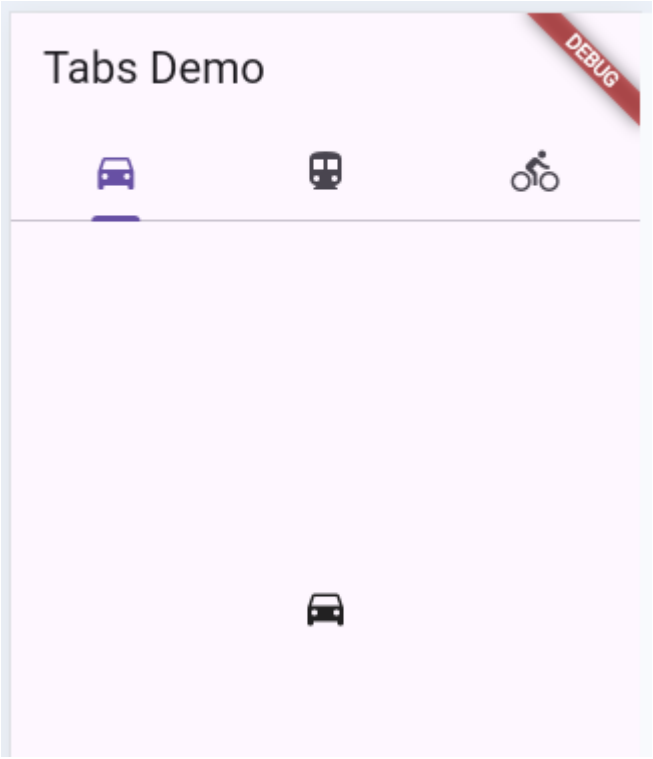
1. User interface

Navigation and Routing: https://docs.flutter.dev/ui/navigation

1.1. Navigation and Routing

Terminology: In Flutter, **screens** and **pages** are called **routes**. The remainder of this recipe refers to routes.

1.1.1. Add tabs to your app




```dart
import 'package:flutter/material.dart';

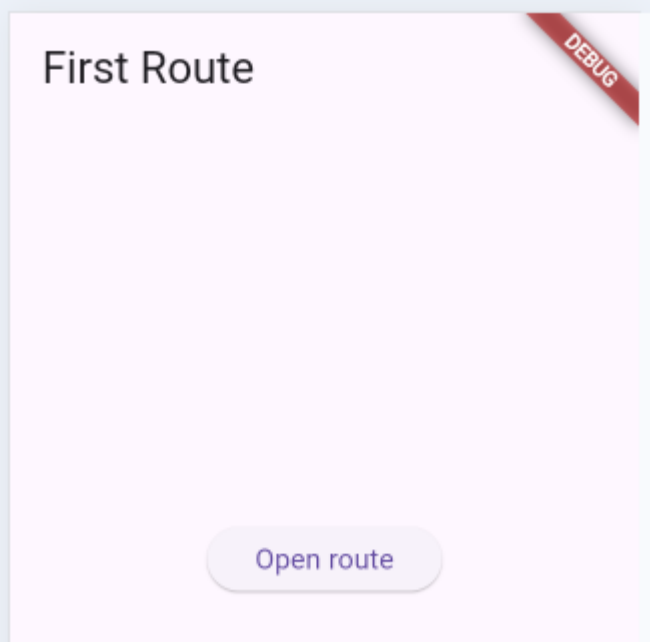
void main() {
  runApp(const TabBarDemo());
}

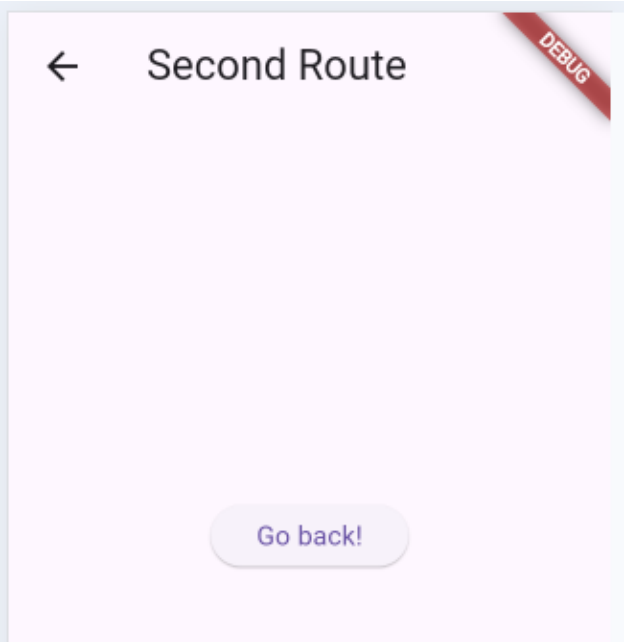
class TabBarDemo extends StatelessWidget {
  const TabBarDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: DefaultTabController(
        length: 3,
        child: Scaffold(
          appBar: AppBar(
            bottom: const TabBar(
```

```
              tabs: [
                Tab(icon: Icon(Icons.directions_car)),
                Tab(icon: Icon(Icons.directions_transit)),
                Tab(icon: Icon(Icons.directions_bike)),
              ],
            ),
            title: const Text('Tabs Demo'),
          ),
          body: const TabBarView(
            // Order is important and must correspond to the order of the tabs
in the TabBar.
            children: [
              Icon(Icons.directions_car),
              Icon(Icons.directions_transit),
              Icon(Icons.directions_bike),
            ],
          ),
        ),
      ),
    );
  }
}
```

1.1.2. Navigate to a new screen and back






```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(title: 'Navigation Basics', home: FirstRoute()));
}

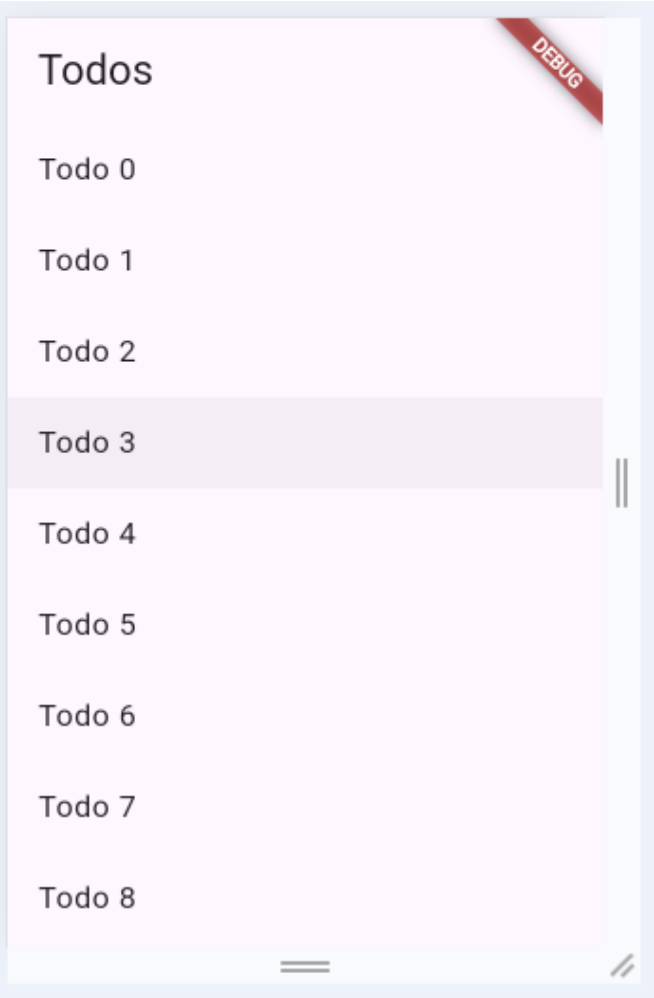
class FirstRoute extends StatelessWidget {
  const FirstRoute({super.key});

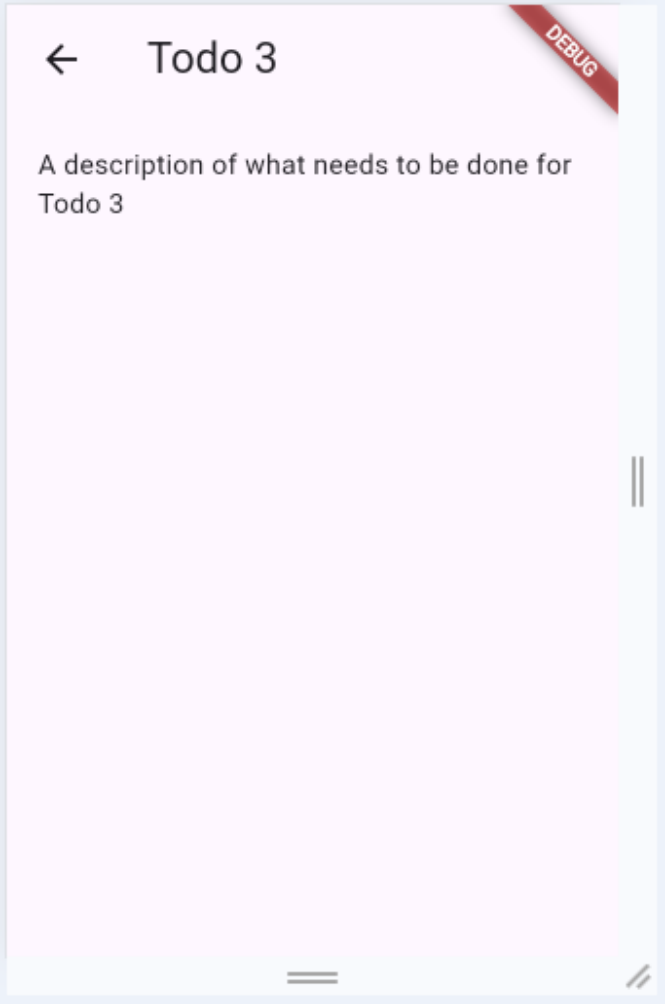
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('First Route')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Open route'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute<void>(
                builder: (context) => const SecondRoute(),
              ),
            );
          },
        ),
      ),
    );
  }
}
```

```dart
class SecondRoute extends StatelessWidget {
  const SecondRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Second Route')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
}
```

1.1.3. Send data to a new screen





Cách 1: Pass data through constructor

```dart
import 'package:flutter/material.dart';

class Todo {
  final String title;
  final String description;

  const Todo(this.title, this.description);
}

void main() {
  runApp(
    MaterialApp(
      title: 'Passing Data',
      home: TodosScreen(
        todos: List.generate(
          20,
          (i) => Todo(
            'Todo $i',
            'A description of what needs to be done for Todo $i',
```

```
          ),
        ),
      ),
    ),
  );
}

class TodosScreen extends StatelessWidget {
  const TodosScreen({super.key, required this.todos});

  final List<Todo> todos;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Todos')),
      body: ListView.builder(
        itemCount: todos.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(todos[index].title),
            // When a user taps the ListTile, navigate to the DetailScreen.
            // Notice that you're not only creating a DetailScreen, you're
            // also passing the current todo through to it.
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute<void>(
                  builder: (context) => DetailScreen(todo: todos[index]),
                ),
              );
            },
          );
        },
      ),
    );
  }
}

class DetailScreen extends StatelessWidget {
  // In the constructor, require a Todo.
  const DetailScreen({super.key, required this.todo});

  // Declare a field that holds the Todo.
  final Todo todo;

  @override
  Widget build(BuildContext context) {
    // Use the Todo to create the UI.
    return Scaffold(
      appBar: AppBar(title: Text(todo.title)),
      body: Padding(
```

```
        padding: const EdgeInsets.all(16),
        child: Text(todo.description),
      ),
    );
  }
}
```

Cách 2: Alternatively, pass the arguments using RouteSettings

```
import 'package:flutter/material.dart';

class Todo {
  final String title;
  final String description;

  const Todo(this.title, this.description);
}

void main() {
  runApp(
    MaterialApp(
      title: 'Passing Data',
      home: TodosScreen(
        todos: List.generate(
          20,
              (i) => Todo(
            'Todo $i',
            'A description of what needs to be done for Todo $i',
          ),
        ),
      ),
    ),
  );
}

class TodosScreen extends StatelessWidget {
  const TodosScreen({super.key, required this.todos});

  final List<Todo> todos;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Todos')),
      body: ListView.builder(
        itemCount: todos.length,
        itemBuilder: (context, index) {
          return ListTile(
            title: Text(todos[index].title),
            // When a user taps the ListTile, navigate to the DetailScreen.
            // Notice that you're not only creating a DetailScreen, you're
```

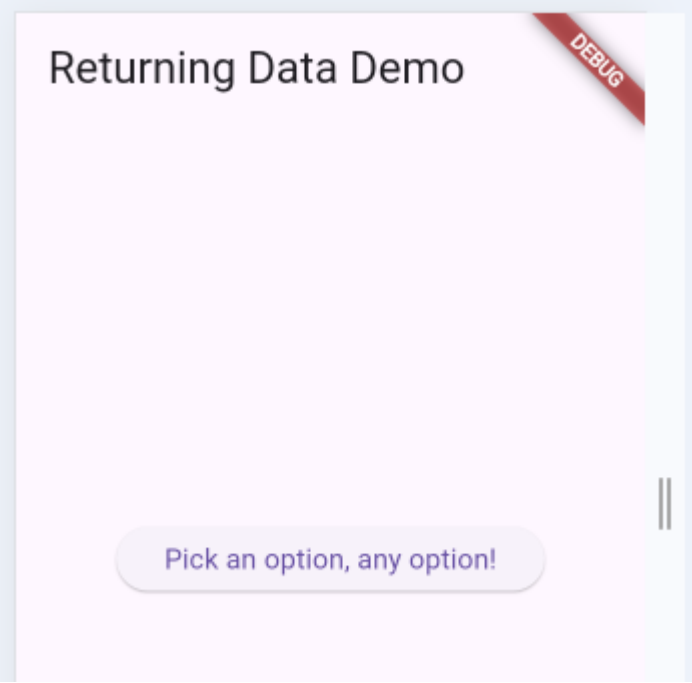
```
            // also passing the current todo through to it.
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute<void>(
                  builder: (context) => const DetailScreen(),
                  // Pass the arguments as part of the RouteSettings. The
                  // DetailScreen reads the arguments from these settings.
                  settings: RouteSettings(arguments: todos[index]),
                ),
              );
            },
          );
        },
      ),
    );
  }
}

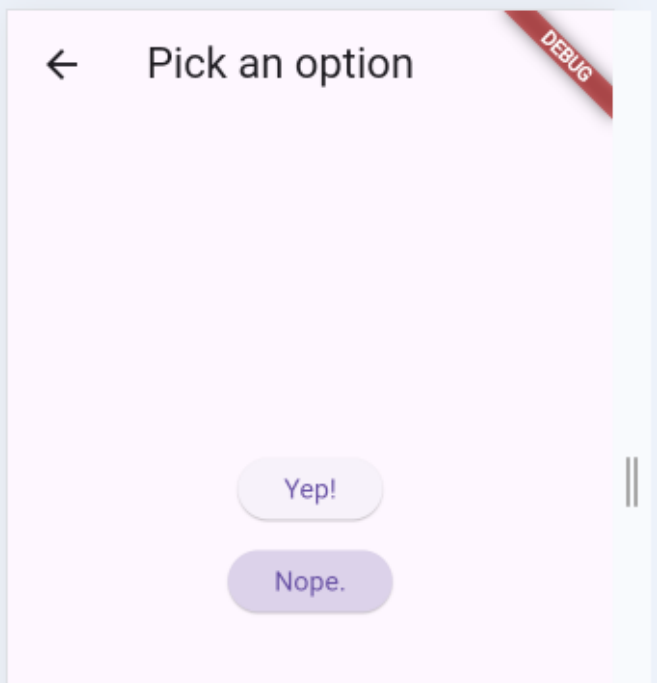
class DetailScreen extends StatelessWidget {
  const DetailScreen({super.key});

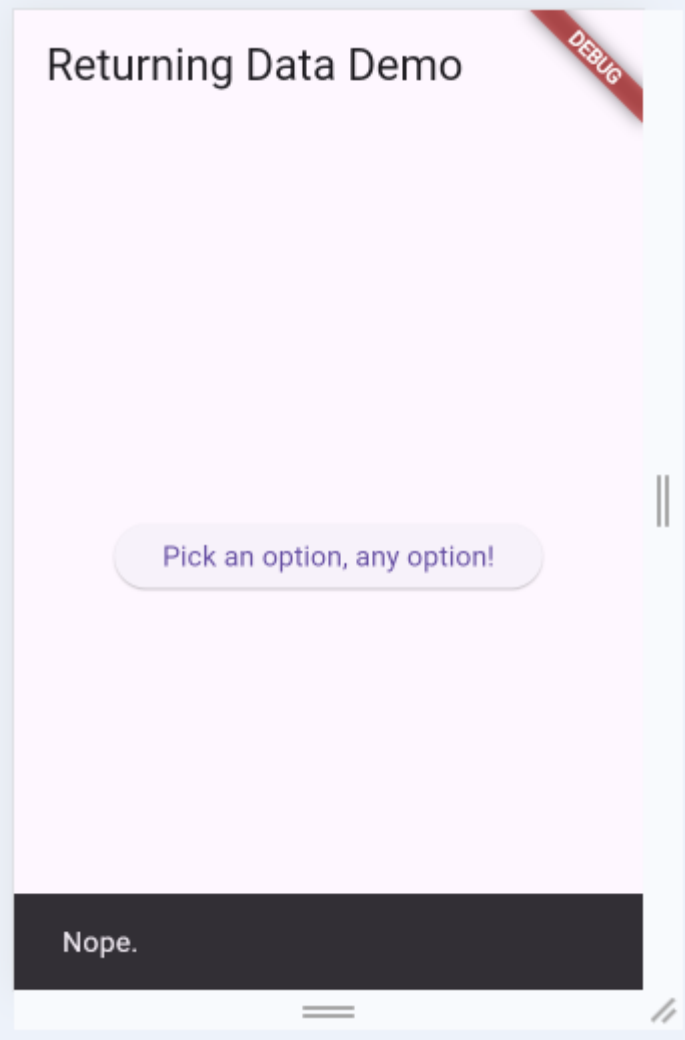
  @override
  Widget build(BuildContext context) {
    final todo = ModalRoute.of(context)!.settings.arguments as Todo;

    // Use the Todo to create the UI.
    return Scaffold(
      appBar: AppBar(title: Text(todo.title)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Text(todo.description),
      ),
    );
  }
}
```

1.1.4. Return data from a screen







```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(title: 'Returning Data', home: HomeScreen()));
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Returning Data Demo')),
      body: const Center(child: SelectionButton()),
    );
  }
}

class SelectionButton extends StatefulWidget {
  const SelectionButton({super.key});

  @override
  State<SelectionButton> createState() => _SelectionButtonState();
}

class _SelectionButtonState extends State<SelectionButton> {
  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        _navigateAndDisplaySelection(context);
      },
      child: const Text('Pick an option, any option!'),
    );
  }

  // A method that launches the SelectionScreen and awaits the result from
  // Navigator.pop.
  Future<void> _navigateAndDisplaySelection(BuildContext context) async {
    // Navigator.push returns a Future that completes after calling
    // Navigator.pop on the Selection Screen.
    final result = await Navigator.push(
      context,
      MaterialPageRoute<String>(builder: (context) => const SelectionScreen()),
    );

    // When a BuildContext is used from a StatefulWidget, the mounted property
    // must be checked after an asynchronous gap.
    if (!context.mounted) return;

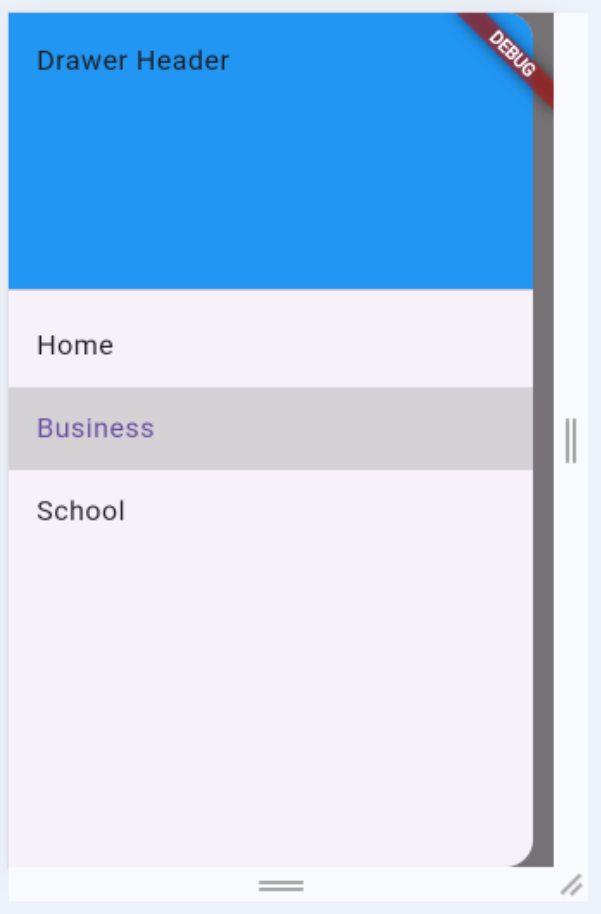
    // After the Selection Screen returns a result, hide any previous snackbars
    // and show the new result.
```

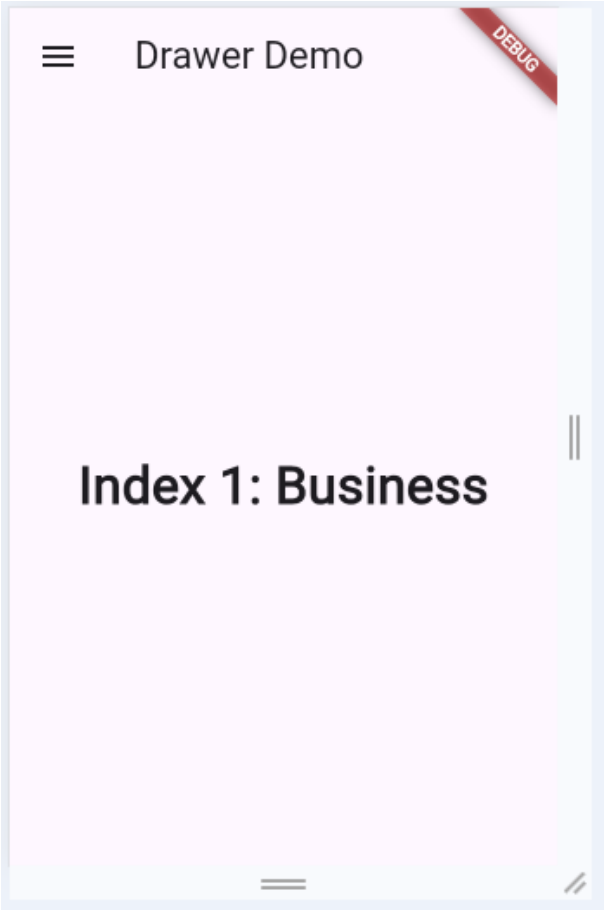
```
    ScaffoldMessenger.of(context)
      ..removeCurrentSnackBar()
      ..showSnackBar(SnackBar(content: Text('$result')));
  }
}

class SelectionScreen extends StatelessWidget {
  const SelectionScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pick an option')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Padding(
              padding: const EdgeInsets.all(8),
              child: ElevatedButton(
                onPressed: () {
                  // Close the screen and return "Yep!" as the result.
                  Navigator.pop(context, 'Yep!');
                },
                child: const Text('Yep!'),
              ),
            ),
            Padding(
              padding: const EdgeInsets.all(8),
              child: ElevatedButton(
                onPressed: () {
                  // Close the screen and return "Nope." as the result.
                  Navigator.pop(context, 'Nope.');
                },
                child: const Text('Nope.'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

1.1.5. Add a drawer to a screen






```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  static const appTitle = 'Drawer Demo';

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: appTitle,
      home: MyHomePage(title: appTitle),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;
```

```
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _selectedIndex = 0;

  static const TextStyle optionStyle = TextStyle(
    fontSize: 30,
    fontWeight: FontWeight.bold,
  );

  static const List<Widget> _widgetOptions = <Widget>[
    Text('Index 0: Home', style: optionStyle),
    Text('Index 1: Business', style: optionStyle),
    Text('Index 2: School', style: optionStyle),
  ];

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
        leading: Builder(
          builder: (context) {
            return IconButton(
              icon: const Icon(Icons.menu),
              onPressed: () {
                Scaffold.of(context).openDrawer();
              },
            );
          },
        ),
      ),

      body: Center(child: _widgetOptions[_selectedIndex]),

      drawer: Drawer(
        // Add a ListView to the drawer. This ensures the user can scroll
        // through the options in the drawer if there isn't enough vertical
        // space to fit everything.
        child: ListView(
          // Important: Remove any padding from the ListView.
          padding: EdgeInsets.zero,
          children: [
            const DrawerHeader(
```

```
              decoration: BoxDecoration(color: Colors.blue),
              child: Text('Drawer Header'),
            ),
            ListTile(
              title: const Text('Home'),
              selected: _selectedIndex == 0,
              onTap: () {
                // Update the state of the app
                _onItemTapped(0);
                // Then close the drawer
                Navigator.pop(context);
              },
            ),
            ListTile(
              title: const Text('Business'),
              selected: _selectedIndex == 1,
              onTap: () {
                // Update the state of the app
                _onItemTapped(1);
                // Then close the drawer
                Navigator.pop(context);
              },
            ),
            ListTile(
              title: const Text('School'),
              selected: _selectedIndex == 2,
              onTap: () {
                // Update the state of the app
                _onItemTapped(2);
                // Then close the drawer
                Navigator.pop(context);
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

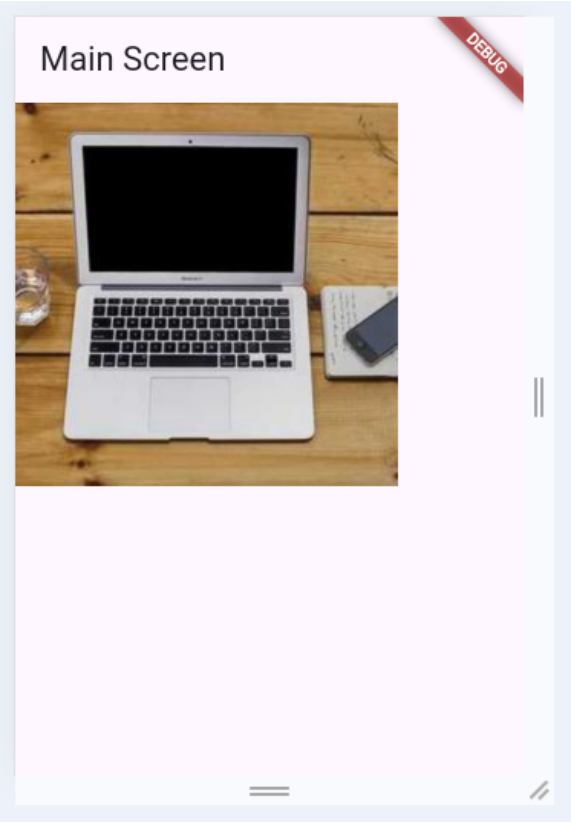
1.1.6. Animate a widget across screens

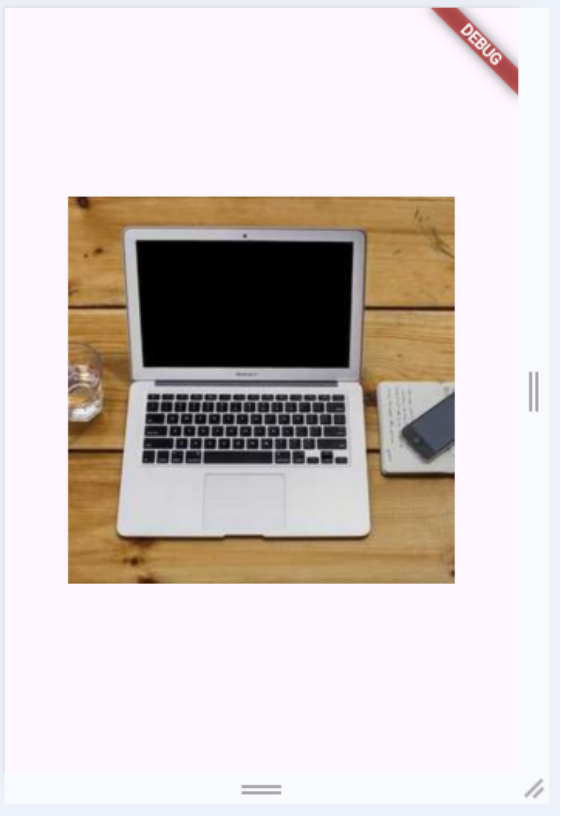
Link: https://docs.flutter.dev/cookbook/navigation/hero-animations

It's often helpful to guide users through an app as they navigate from screen to screen. A common technique to lead users through an app is to animate a widget from one screen to the next. This creates a visual anchor connecting the two screens.

Use the `Hero` widget to animate a widget from one screen to the next. This recipe uses the following steps:

- Create two screens showing the same image.
- Add a `Hero` widget to the first screen.
- Add a `Hero` widget to the second screen.






```dart
import 'package:flutter/material.dart';

void main() => runApp(const HeroApp());

class HeroApp extends StatelessWidget {
  const HeroApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(title: 'Transition Demo', home: MainScreen());
  }
}

class MainScreen extends StatelessWidget {
```

```dart
  const MainScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Main Screen')),
      body: GestureDetector(
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute<void>(
              builder: (context) {
                return const DetailScreen();
              },
            ),
          );
        },
        child: Hero(
          tag: 'imageHero', // tag: an object that identifies the Hero. It must be the same on both screens.
          child: Image.network('https://picsum.photos/250?image=9'), // The widget to animate across screens.
        ),
      ),
    );
  }
}

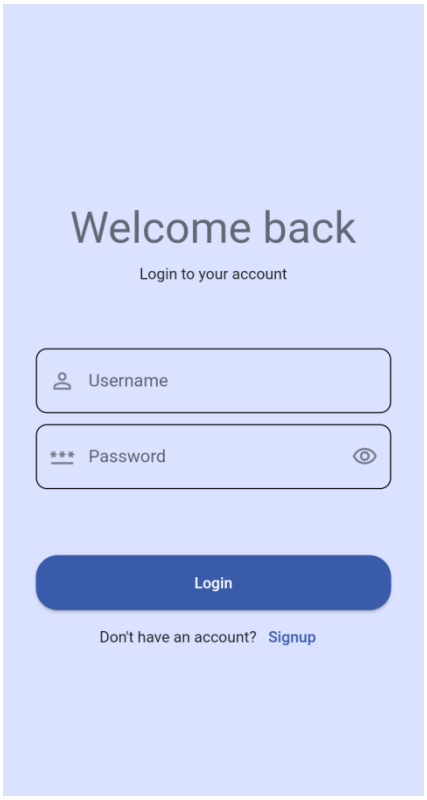
class DetailScreen extends StatelessWidget {
  const DetailScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: GestureDetector(
        onTap: () {
          Navigator.pop(context);
        },
        child: Center(
          child: Hero(
            tag: 'imageHero',
            child: Image.network('https://picsum.photos/250?image=9'),
          ),
        ),
      ),
    );
  }
}
```

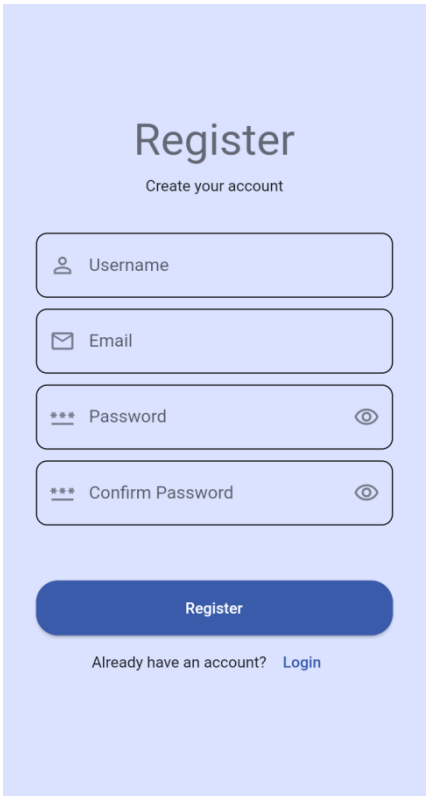
2. Thực hành

Bài tập 1. Login and Registration

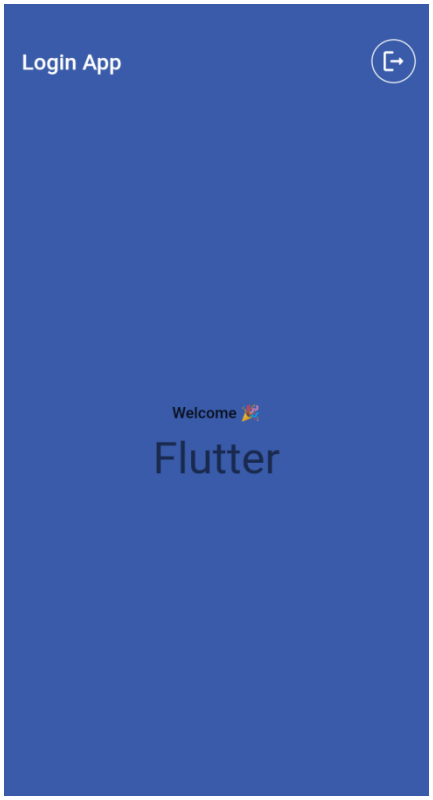
Task 1: Build the Login function including screens as shown below.



Login screen



Signup screen

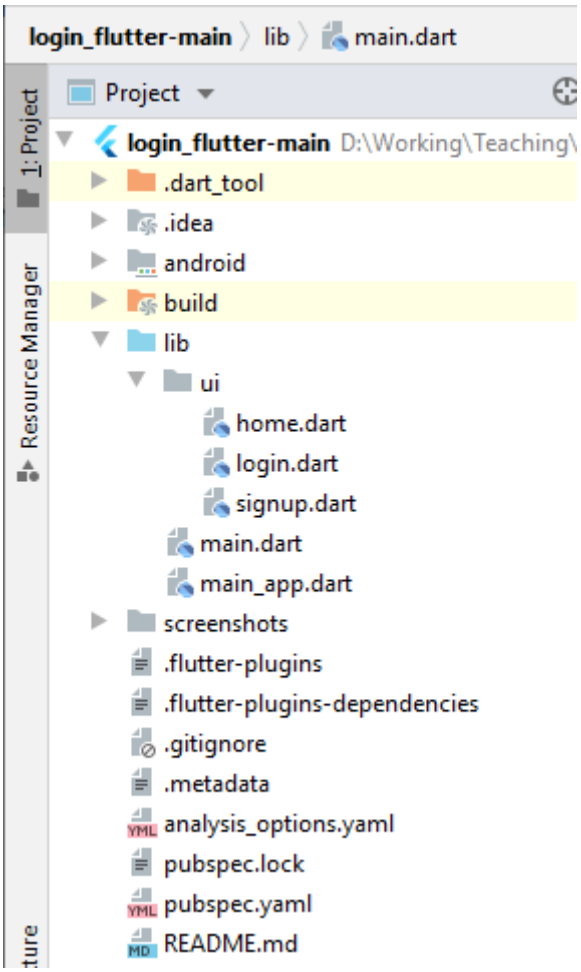


Home screen

Task 2: Ở màn hình Signup, giả sử mật khẩu yêu cầu ít nhất 8 ký tự, và phải chứa ký số, ký tự đặc biệt. Hãy thực hiện chức năng hiển thị trạng thái mật khẩu, cho biết mật khẩu đã đủ mạnh (thỏa yêu cầu) hay chưa ngay khi người dùng đang nhập mật khẩu.

Project structure

Path	Description
lib/ui/home.dart	Home screen
lib/ui/login.dart	Login screen
lib/ui/signup.dart	Signup screen
lib/main.dart	The entry point for the application, contains the main() function.
lib/main_app.dart	Contains the MaterialApp (build screens).



File login.dart

```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';

import 'home.dart';
import 'signup.dart';

class Login extends StatefulWidget {
  const Login({
    Key? key,
  }) : super(key: key);

  @override
  State<Login> createState() => _LoginState();
}

class _LoginState extends State<Login> {
  final GlobalKey<FormState> _formKey = GlobalKey();

  final FocusNode _focusNodePassword = FocusNode();
  final TextEditingController _controllerUsername = TextEditingController();
  final TextEditingController _controllerPassword = TextEditingController();

  bool _obscurePassword = true; // indicate that showing/hiding password.

  final Box _boxLogin = Hive.box("login"); // Like a session variable to
  // store your login status (true/false) and login name

  final Box _boxAccounts = Hive.box("accounts"); // Like your database to
  // store pairs of usename and password

  @override
  Widget build(BuildContext context) {
    if (_boxLogin.get("loginStatus") ?? false) {
      // ?? nghĩa là: nếu về trái bị null, lấy trả trị bên phải dấu ??
      return Home();
    }

    return Scaffold(
      backgroundColor: Theme.of(context).colorScheme.primaryContainer,
      body: Form(
        key: _formKey,
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(30.0),
          child: Column(
            children: [
              const SizedBox(height: 150),
              Text(
                "Welcome back",
                style: Theme.of(context).textTheme.headlineLarge,
              ),
```

```
              const SizedBox(height: 10),
              Text(
                "Login to your account",
                style: Theme.of(context).textTheme.bodyMedium,
              ),
              const SizedBox(height: 60),
              TextFormField(
                controller: _controllerUsername,
                keyboardType: TextInputType.name,
                decoration: InputDecoration(
                  labelText: "Username",
                  prefixIcon: const Icon(Icons.person_outline),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(0),
                  ),
                  enabledBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
                onEditingComplete: () => _focusNodePassword.requestFocus(),
                validator: (String? value) {
                  if (value == null || value.isEmpty) {
                    return "Please enter username.";
                  } else if (!_boxAccounts.containsKey(value)) {
                    return "Username is not registered.";
                  }

                  return null;
                },
              ),
              const SizedBox(height: 10),
              TextFormField(
                controller: _controllerPassword,
                focusNode: _focusNodePassword,
                obscureText: _obscurePassword,
                keyboardType: TextInputType.visiblePassword,
                decoration: InputDecoration(
                  labelText: "Password",
                  prefixIcon: const Icon(Icons.password_outlined),
                  suffixIcon: IconButton(
                      onPressed: () {
                        setState(() {
                          _obscurePassword = !_obscurePassword;
                        });
                      },
                      icon: _obscurePassword
                          ? const Icon(Icons.visibility_outlined)
                          : const Icon(Icons.visibility_off_outlined)),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(0),
                  ),
                  enabledBorder: OutlineInputBorder(
```

```
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
                validator: (String? value) {
                  if (value == null || value.isEmpty) {
                    return "Please enter password.";
                  } else if (value !=
                      _boxAccounts.get(_controllerUsername.text)) {
                    return "Wrong password.";
                  }

                  return null;
                },
              ),
              const SizedBox(height: 60),
              Column(
                children: [
                  ElevatedButton(
                    style: ElevatedButton.styleFrom(
                      minimumSize: const Size.fromHeight(50),
                      shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(20),
                      ),
                    ),
                    onPressed: () {
                      if (_formKey.currentState?.validate() ?? false) {
                        _boxLogin.put("loginStatus", true);
                        _boxLogin.put("userName", _controllerUsername.text);

                        Navigator.pushReplacement(
                          context,
                          MaterialPageRoute(
                            builder: (context) {
                              return Home();
                            },
                          ),
                        );
                      }
                    },
                    child: const Text("Login"),
                  ),
                  Row(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      const Text("Don't have an account?"),
                      TextButton(
                        onPressed: () {
                          _formKey.currentState?.reset();

                          Navigator.push(
                            context,
                            MaterialPageRoute(
```

```
                                builder: (context) {
                                  return const Signup();
                                },
                              ),
                            );
                          },
                          child: const Text("Signup"),
                        ),
                      ],
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      );
  }

  @override
  void dispose() {
    _focusNodePassword.dispose();
    _controllerUsername.dispose();
    _controllerPassword.dispose();
    super.dispose();
  }
}
```

File signup.dart

```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';

class Signup extends StatefulWidget {
  const Signup({super.key});

  @override
  State<Signup> createState() => _SignupState();
}

class _SignupState extends State<Signup> {
  final GlobalKey<FormState> _formKey = GlobalKey();

  final FocusNode _focusNodeEmail = FocusNode();
  final FocusNode _focusNodePassword = FocusNode();
  final FocusNode _focusNodeConfirmPassword = FocusNode();
  final TextEditingController _controllerUsername = TextEditingController();
  final TextEditingController _controllerEmail = TextEditingController();
  final TextEditingController _controllerPassword = TextEditingController();
  final TextEditingController _controllerConFirmPassword =
      TextEditingController();

  final Box _boxAccounts = Hive.box("accounts");
  bool _obscurePassword = true;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Theme.of(context).colorScheme.primaryContainer,
      body: Form(
        key: _formKey,
        child: SingleChildScrollView(
          padding: const EdgeInsets.symmetric(horizontal: 30.0),
          child: Column(
            children: [
              const SizedBox(height: 100),
              Text(
                "Register",
                style: Theme.of(context).textTheme.headlineLarge,
              ),
              const SizedBox(height: 10),
              Text(
                "Create your account",
                style: Theme.of(context).textTheme.bodyMedium,
              ),
              const SizedBox(height: 35),
              TextFormField(
                controller: _controllerUsername,
                keyboardType: TextInputType.name,
                decoration: InputDecoration(
```

```
                labelText: "Username",
                prefixIcon: const Icon(Icons.person_outline),
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
                enabledBorder: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
              ),
              validator: (String? value) {
                if (value == null || value.isEmpty) {
                  return "Please enter username.";
                } else if (_boxAccounts.containsKey(value)) {
                  return "Username is already registered.";
                }

                return null;
              },
              onEditingComplete: () => _focusNodeEmail.requestFocus(),
               // focus on Email after editting finished.
            ),
            const SizedBox(height: 10),
            TextFormField(
              controller: _controllerEmail,
              focusNode: _focusNodeEmail,
              keyboardType: TextInputType.emailAddress,
              decoration: InputDecoration(
                labelText: "Email",
                prefixIcon: const Icon(Icons.email_outlined),
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
                enabledBorder: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10),
                ),
              ),
              validator: (String? value) {
                if (value == null || value.isEmpty) {
                  return "Please enter email.";
                } else if (!(value.contains('@') && value.contains('.'))) {
                  return "Invalid email";
                }
                return null;
              },
              onEditingComplete: () => _focusNodePassword.requestFocus(),
            ),
            const SizedBox(height: 10),
            TextFormField(
              controller: _controllerPassword,
              obscureText: _obscurePassword,
              focusNode: _focusNodePassword,
              keyboardType: TextInputType.visiblePassword,
```

```
                decoration: InputDecoration(
                  labelText: "Password",
                  prefixIcon: const Icon(Icons.password_outlined),
                  suffixIcon: IconButton(
                      onPressed: () {
                        setState(() {
                          _obscurePassword = !_obscurePassword;
                        });
                      },
                      icon: _obscurePassword
                          ? const Icon(Icons.visibility_outlined)
                          : const Icon(Icons.visibility_off_outlined)),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(10),
                  ),
                  enabledBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
                validator: (String? value) {
                  if (value == null || value.isEmpty) {
                    return "Please enter password.";
                  } else if (value.length < 8) {
                    return "Password must be at least 8 character.";
                  }
                  return null;
                },
                onEditingComplete: () =>
                    _focusNodeConfirmPassword.requestFocus(),
              ),
              const SizedBox(height: 10),
              TextFormField(
                controller: _controllerConFirmPassword,
                obscureText: _obscurePassword,
                focusNode: _focusNodeConfirmPassword,
                keyboardType: TextInputType.visiblePassword,
                decoration: InputDecoration(
                  labelText: "Confirm Password",
                  prefixIcon: const Icon(Icons.password_outlined),
                  suffixIcon: IconButton(
                      onPressed: () {
                        setState(() {
                          _obscurePassword = !_obscurePassword;
                        });
                      },
                      icon: _obscurePassword
                          ? const Icon(Icons.visibility_outlined)
                          : const Icon(Icons.visibility_off_outlined)),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(10),
                  ),
                  enabledBorder: OutlineInputBorder(
```

```
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
                validator: (String? value) {
                  if (value == null || value.isEmpty) {
                    return "Please enter password.";
                  } else if (value != _controllerPassword.text) {
                    return "Password doesn't match.";
                  }
                  return null;
                },
              ),
              const SizedBox(height: 50),
              Column(
                children: [
                  ElevatedButton(
                    style: ElevatedButton.styleFrom(
                      minimumSize: const Size.fromHeight(50),
                      shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(20),
                      ),
                    ),
                    onPressed: () {
                      if (_formKey.currentState?.validate() ?? false) {
                        _boxAccounts.put(
                          _controllerUsername.text,
                          _controllerConFirmPassword.text,
                        );

                        ScaffoldMessenger.of(context).showSnackBar(
                          SnackBar(
                            width: 200,
                            backgroundColor:
                                Theme.of(context).colorScheme.secondary,
                            shape: RoundedRectangleBorder(
                              borderRadius: BorderRadius.circular(10),
                            ),
                            behavior: SnackBarBehavior.floating,
                            content: const Text("Registered Successfully"),
                          ),
                        );

                        _formKey.currentState?.reset();

                        Navigator.pop(context);
                      }
                    },
                    child: const Text("Register"),
                  ),
                  Row(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
```

```
                        const Text("Already have an account?"),
                        TextButton(
                          onPressed: () => Navigator.pop(context),
                          child: const Text("Login"),
                        ),
                      ],
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }

  @override
  void dispose() {
    _focusNodeEmail.dispose();
    _focusNodePassword.dispose();
    _focusNodeConfirmPassword.dispose();
    _controllerUsername.dispose();
    _controllerEmail.dispose();
    _controllerPassword.dispose();
    _controllerConFirmPassword.dispose();
    super.dispose();
  }
}
```

File home.dart

```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';

import 'login.dart';

class Home extends StatelessWidget {
  Home({super.key});

  final Box _boxLogin = Hive.box("login");

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Login App"),
        elevation: 10,
        actions: [
          Padding(
            padding: const EdgeInsets.all(8.0),
            child: DecoratedBox(
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                border: Border.all(color: Colors.white),
              ),
              child: IconButton(
                onPressed: () {
                  _boxLogin.clear();
                  _boxLogin.put("loginStatus", false);
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(
                      builder: (context) {
                        return const Login();
                      },
                    ),
                  );
                },
                icon: const Icon(Icons.logout_rounded),
              ),
            ),
          )
        ],
      ),
      backgroundColor: Theme.of(context).colorScheme.primary,
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              "Welcome 🎉",
```

```
              style: Theme.of(context).textTheme.bodyLarge,
            ),
            const SizedBox(height: 10),
            Text(
              _boxLogin.get("userName"),
              style: Theme.of(context).textTheme.headlineLarge,
            ),
          ],
        ),
      ),
    );
  }
}
```

File main.dart

```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';

import 'main_app.dart';

void main() async {
  await _initHive();
  runApp(const MainApp());
}

Future<void> _initHive() async {
  // Hive is a lightweight and blazing fast key-value database written in pure
Dart.
  // Ref: https://pub.dev/packages/hive
  await Hive.initFlutter();
  await Hive.openBox("login");
  await Hive.openBox("accounts");
}

File main_app.dart
import 'package:flutter/material.dart';

import 'ui/login.dart';

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.from(
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color.fromRGBO(32, 63, 129, 1.0),
        ),
```

```
      ),
      home: const Login(),
    );
  }
}
```

Bài tập 2. Login and Registration với GoRouter package (đọc thêm)

Yêu cầu:

Viết ứng dụng gồm các màn hình:

- Trang chủ, có nút Login nếu chưa đăng nhập. Nếu đã đăng nhập, hiển thị nút Logout.
- Trang Login, nhập Email, Password. Nếu chưa có tài khoản, người dùng bấm vào nút Register. Nếu quên mật khẩu, bấm vào nút Forgot password.
- Trang Register, yêu cầu nhập Email, Password, Confirm password. Khi đăng nhập thành công, chuyển về trang chủ, có nút Logout.

Hướng dẫn:

Cấu trúc project:

```
lib/
│
├── main.dart
├── app_router.dart                 # Định nghĩa go_router và redirect logic
│
├── services/
│   └── auth_service.dart           # Giả lập dịch vụ đăng nhập
│
├── views/
│   ├── home_page.dart
│   ├── login_page.dart
│   └── register_page.dart
│
└── widgets/
    └── custom_text_field.dart      # Tái sử dụng cho login/register
```

File: lib/services/auth_service.dart

```
import 'package:flutter/foundation.dart';

/// Giả lập trạng thái đăng nhập
class AuthService extends ChangeNotifier {
  bool _isLoggedIn = false;

  bool get isLoggedIn => _isLoggedIn;

  void login(String email, String password) {
    // Giả lập đăng nhập thành công
    _isLoggedIn = true;
    notifyListeners();
  }

  void logout() {
    _isLoggedIn = false;
    notifyListeners();
  }

  void register(String email, String password) {
    // Giả lập đăng ký (ở thực tế sẽ gọi API)
    _isLoggedIn = true;
    notifyListeners();
  }
}
```

File: lib/app_router.dart

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'services/auth_service.dart';
import 'views/home_page.dart';
import 'views/login_page.dart';
import 'views/register_page.dart';

GoRouter createRouter(AuthService authService) {
  return GoRouter(
    initialLocation: '/',
    refreshListenable: authService, // rebuild router khi login/logout
    redirect: (context, state) {
      final loggedIn = authService.isLoggedIn;
      final loggingIn = state.matchedLocation == '/login' || state.matchedLocation ==
'/register';

      // Nếu chưa login mà không ở trang login/register → chuyển đến login
      if (!loggedIn && !loggingIn) return '/login';

      // Nếu đã login mà đang ở trang login/register → chuyển về home
      if (loggedIn && loggingIn) return '/';

      return null;
    },
    routes: [
      GoRoute(
        path: '/',
        builder: (context, state) => const HomePage(),
      ),
      GoRoute(
        path: '/login',
        builder: (context, state) => const LoginPage(),
      ),
      GoRoute(
        path: '/register',
        builder: (context, state) => const RegisterPage(),
      ),
    ],
  );
}
```

File: lib/views/home_page.dart

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import '../services/auth_service.dart';
import 'package:provider/provider.dart';

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    final auth = context.watch<AuthService>();

    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: auth.isLoggedIn
            ? ElevatedButton(
                onPressed: auth.logout,
                child: const Text('Logout'),
              )
            : ElevatedButton(
                onPressed: () => context.go('/login'),
                child: const Text('Login'),
              ),
      ),
    );
  }
}
```

File: lib/views/login_page.dart

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:provider/provider.dart';
import '../services/auth_service.dart';
import '../widgets/custom_text_field.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    final auth = context.read<AuthService>();

    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            CustomTextField(controller: _emailController, label: 'Email'),
            const SizedBox(height: 12),
            CustomTextField(controller: _passwordController, label: 'Password',
obscureText: true),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () {
                auth.login(_emailController.text, _passwordController.text);
              },
              child: const Text('Login'),
            ),
            TextButton(
              onPressed: () => context.go('/register'),
              child: const Text('Register new account'),
            ),
            TextButton(
              onPressed: () {
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Forgot password clicked')),
                );
              },
              child: const Text('Forgot password?'),
            ),
          ],
        ),
      ),
    );
```

```
    }
}
```

File: lib/views/register_page.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../services/auth_service.dart';
import '../widgets/custom_text_field.dart';

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});

  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    final auth = context.read<AuthService>();

    return Scaffold(
      appBar: AppBar(title: const Text('Register')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            CustomTextField(controller: _emailController, label: 'Email'),
            const SizedBox(height: 12),
            CustomTextField(controller: _passwordController, label: 'Password',
obscureText: true),
            const SizedBox(height: 12),
            CustomTextField(controller: _confirmController, label: 'Confirm Password',
obscureText: true),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () {
                if (_passwordController.text != _confirmController.text) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(content: Text('Passwords do not match')),
                  );
                  return;
                }
                auth.register(_emailController.text, _passwordController.text);
              },
              child: const Text('Register'),
            ),
          ],
        ),
      ),
    );
  }
}
```

File: lib/widgets/custom_text_field.dart

```
import 'package:flutter/material.dart';

class CustomTextField extends StatelessWidget {
  final TextEditingController controller;
  final String label;
  final bool obscureText;

  const CustomTextField({
    super.key,
    required this.controller,
    required this.label,
    this.obscureText = false,
  });

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: controller,
      obscureText: obscureText,
      decoration: InputDecoration(
        labelText: label,
        border: const OutlineInputBorder(),
      ),
    );
  }
}
```

File: lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'app_router.dart';
import 'services/auth_service.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => AuthService(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final authService = context.watch<AuthService>();
    final router = createRouter(authService);

    return MaterialApp.router(
      title: 'Bookstore Demo',
      routerConfig: router,
```

```
      theme: ThemeData(primarySwatch: Colors.indigo),
    );
  }
}
```

Giải thích:

Thành phần	Vai trò
go_router	Quản lý điều hướng, có thể redirect logic khi login/logout
AuthService	Lưu trạng thái đăng nhập (giả lập ChangeNotifier)
Provider	Cho phép các widget con truy cập AuthService
context.watch<AuthService>()	Tự động rebuild khi trạng thái login thay đổi
GoRouter với redirect:	Giúp điều hướng tự động dựa vào login state
context.go('/path')	Điều hướng đến route tương ứng

Bài tập 3. How to Get Started With GoRouter in Flutter (đọc thêm)

https://www.freecodecamp.org/news/how-to-get-started-with-gorouter-in-flutter/

```
go_router_project/
|-- lib/
|   |-- main.dart
|   |-- models/
|   |   |-- product.dart
|   |-- controller/
|   |   |-- product_controller.dart
|   |-- config/
|   |   |-- route_config.dart
|   |-- screens/
|   |   |-- product_details_screen.dart
|   |   |-- product_list_screen.dart
|   |   |-- product_purchase_screen.dart
|   |-- widgets/
|       |-- bottom_container.dart
|       |-- color_container.dart
|       |-- ratings.dart
|       |-- search_section.dart
|       |-- show_modal.dart
|       |-- single_product.dart
|-- pubspec.yaml
```